

Support Vector Machine

MAKINI CALEB

2026-10-05

SETTING A WORKING DIRECTORY

```
setwd("C:/Users/ADMIN/OneDrive/Desktop/caleb makini")
```

load libraries

```
library(dplyr)
library(tidyverse)
library(e1071)
library(caret)
library(psych)
library(ggplot2)
```

support vector machine(SVM)

- is a supervised machine learning algorithm used for both classification and regression task.
- svm works by finding optimal boundary (hyperplane) that best separates data points into different classes while maximising the margin between them
-

load data set

```
data(mtcars)
head(data)
```

```
1 function (... , list = character(), package = NULL, lib.loc = NULL,
2     verbose = getOption("verbose"), envir = .GlobalEnv, overwrite =
TRUE)
3 {
4     fileExt <- function(x) {
5         db <- grepl("\\\\.([^.]+\\\\\\. (gz|bz2|xz))$", x)
6         ans <- sub("\\.([^.]+\\\\\\. (gz|bz2|xz))$", "", x)
```

check structure

```
str(mtcars)
```

```
'data.frame':  32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num   6  6  4  6  8  6  8  4  4  6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110  93 110 175 105 245  62  95 123 ...
 $ drat: num   3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num   2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num   16.5 17 18.6 19.4 17 ...
 $ vs  : num   0  0  1  1  0  1  0  1  1  1 ...
 $ am  : num   1  1  1  0  0  0  0  0  0  0 ...
 $ gear: num    4  4  4  3  3  3  3  4  4  4 ...
 $ carb: num    4  4  1  1  2  1  4  2  2  4 ...
```

summary statistics

```
describe(mtcars)
```

	vars	n	mean	sd	median	trimmed	mad	min	max	range
skew										
mpg	1	32	20.09	6.03	19.20	19.70	5.41	10.40	33.90	23.50
0.61										
cyl	2	32	6.19	1.79	6.00	6.23	2.97	4.00	8.00	4.00
0.17										
disp	3	32	230.72	123.94	196.30	222.52	140.48	71.10	472.00	400.90
0.38										
hp	4	32	146.69	68.56	123.00	141.19	77.10	52.00	335.00	283.00
0.73										
drat	5	32	3.60	0.53	3.70	3.58	0.70	2.76	4.93	2.17
0.27										
wt	6	32	3.22	0.98	3.33	3.15	0.77	1.51	5.42	3.91
0.42										
qsec	7	32	17.85	1.79	17.71	17.83	1.42	14.50	22.90	8.40
0.37										
vs	8	32	0.44	0.50	0.00	0.42	0.00	0.00	1.00	1.00
0.24										
am	9	32	0.41	0.50	0.00	0.38	0.00	0.00	1.00	1.00
0.36										
gear	10	32	3.69	0.74	4.00	3.62	1.48	3.00	5.00	2.00
0.53										
carb	11	32	2.81	1.62	2.00	2.65	1.48	1.00	8.00	7.00
1.05										
kurtosis			se							
mpg			-0.37	1.07						
cyl			-1.76	0.32						
disp			-1.21	21.91						
hp			-0.14	12.12						
drat			-0.71	0.09						
wt			-0.02	0.17						
qsec			0.34	0.32						
vs			-2.00	0.09						
am			-1.92	0.09						
gear			-1.07	0.13						
carb			1.26	0.29						

check missing values

```
colSums((is.na(mtcars)))
```

mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	0	0	0	0	0	0	0	0	0	0

split data

```
set.seed(123)
split_index <- createDataPartition(mtcars$mpg, p = 0.8, list = FALSE)
train_data1 <- mtcars[split_index, ]
test_data1 <- mtcars[-split_index, ]
```

data dimension

```
dim(train_data1)
```

```
[1] 28 11
```

```
dim(test_data1)
```

```
[1] 4 11
```

scale the predictor variables

```
x_train <- train_data1[, -1]
y_train <- train_data1$mpg
x_test <- test_data1[, -1]
y_test <- test_data1$mpg
preProc <- preProcess(x_train, method = c("center", "scale"))
x_train_scaled <- predict(preProc, x_train)
x_test_scaled <- predict(preProc, x_test)
```

train SVM model

```
svm_model <- svm(x = x_train_scaled, y = y_train, type = "eps-regression", kernel = "radial")
print(svm_model)
```

```
Call:
svm.default(x = x_train_scaled, y = y_train, type = "eps-regression",
            kernel = "radial")
```

```
Parameters:
  SVM-Type:  eps-regression
  SVM-Kernel: radial
    cost:    1
   gamma:   0.1
  epsilon:  0.1
```

```
Number of Support Vectors: 27
```

make predictions

```
predictions <- predict(svm_model, newdata = x_test_scaled)
```

evaluate model performance

rmse

```
rmse <- sqrt(mean((predictions - y_test)^2))
rmse
```

```
[1] 3.699094
```

mae

```
mae <- mean(abs(predictions - y_test))
mae
```

```
[1] 2.58204
```

r2

```
R2 = cor(predictions, y_test)^2
R2
```

```
[1] 0.978711
```

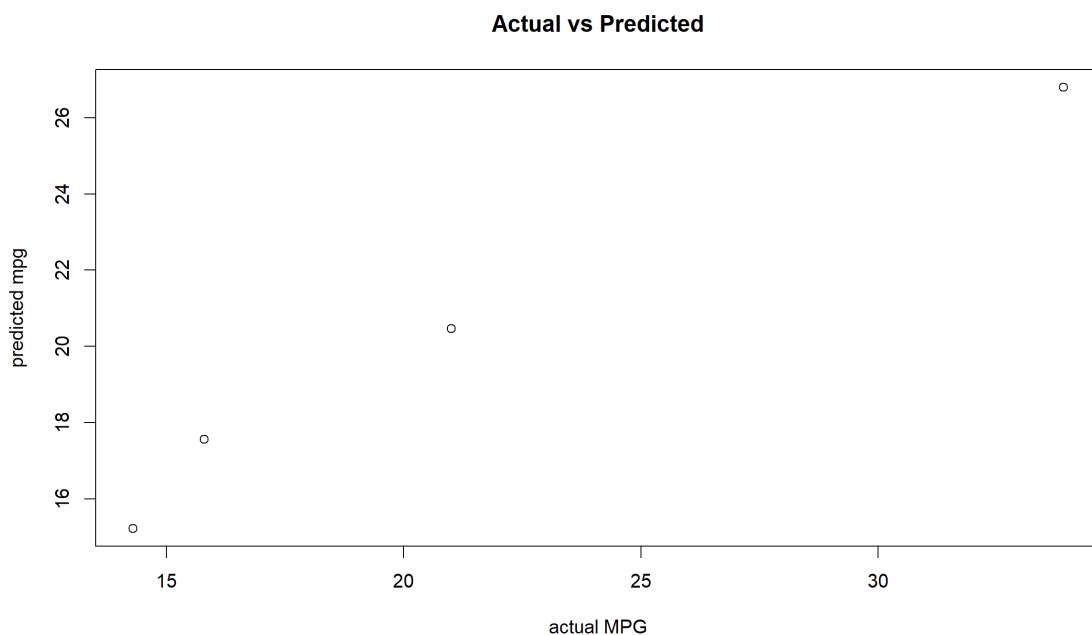
actual vs predicted values

```
results <- data.frame(actual = y_test, predicted = round(predictions,  
2))  
results
```

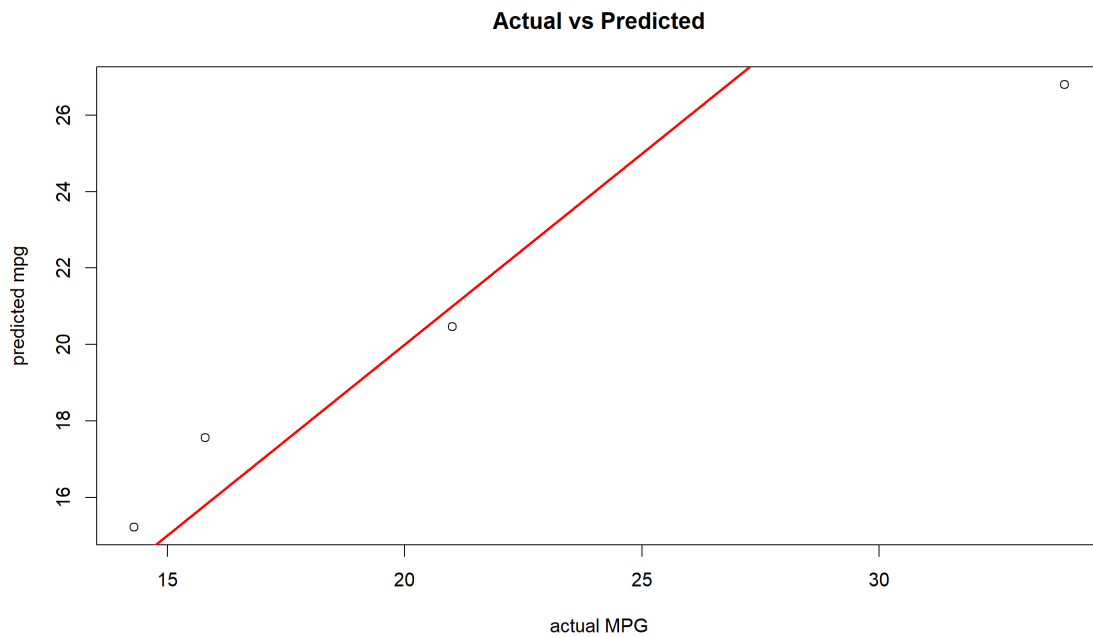
	actual	predicted
Mazda RX4 Wag	21.0	20.46
Duster 360	14.3	15.23
Toyota Corolla	33.9	26.79
Ford Pantera L	15.8	17.56

scatterplot of predictions

```
plot(y_test, predictions, main = "Actual vs Predicted", xlab = "actual  
MPG", ylab = "predicted mpg")
```



```
abline(0, 1, col = "red", lwd = 2)
```



hyperparamater tuning for better accuracy

```
set.seed(123)
turned_model <- tune(svm, mpg ~ ., data = train_data1, kernel = "radial",
  ranges = list(cost = c(0.1, 1, 10, 100), gamma = c(0.001, 0.01, 0.1, 1)))
summary(turned_model)
```

Parameter tuning of 'svm':

- sampling method: 10-fold cross validation

- best parameters:

```
cost gamma  
100 0.001
```

- best performance: 9.829934

- Detailed performance results:

	cost	gamma	error	dispersion
1	0.1	0.001	37.207595	46.465331
2	1.0	0.001	26.084064	35.232023
3	10.0	0.001	9.967809	15.034870
4	100.0	0.001	9.829934	11.250340
5	0.1	0.010	28.126860	37.680298
6	1.0	0.010	11.118237	16.296164
7	10.0	0.010	10.959620	12.349576
8	100.0	0.010	15.020004	9.349671
9	0.1	0.100	24.267188	34.201221
10	1.0	0.100	15.641217	20.443995
11	10.0	0.100	13.658459	14.380619
12	100.0	0.100	23.175968	28.441952
13	0.1	1.000	34.430218	43.184000
14	1.0	1.000	21.652496	28.751322
15	10.0	1.000	18.339938	23.208573
16	100.0	1.000	18.339938	23.208573

best model

```
best_svm <- turned_model$best.model  
pred_best <- predict(best_svm, test_data1)  
rmse_best <- sqrt(mean((pred_best - test_data1$mpg)^2))  
mae_best <- mean(abs(pred_best - test_data1$mpg))  
rmse_best
```

```
[1] 3.815788
```

```
mae_best
```

```
[1] 2.733657
```

SVM CLASSIFICATION

```
data(iris)
set.seed(123)
train_data2 <- createDataPartition(iris$Species, p = 0.8, list = FALSE)
train <- iris[train_data2, ]
test <- iris[-train_data2, ]
svm_classifier <- svm(Species ~ ., data = train, kernel = "radial")
pred_class <- predict(svm_classifier, test)
confusionMatrix(pred_class, test$Species)
```

Confusion Matrix and Statistics

	Reference		
Prediction	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	10	2
virginica	0	0	8

Overall Statistics

Accuracy : 0.9333
95% CI : (0.7793, 0.9918)
No Information Rate : 0.3333
P-Value [Acc > NIR] : 8.747e-12

Kappa : 0.9

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: setosa	Class: versicolor	Class: virginica
Sensitivity	1.0000	1.0000	0.8000
Specificity	1.0000	0.9000	1.0000
Pos Pred Value	1.0000	0.8333	1.0000
Neg Pred Value	1.0000	1.0000	0.9091
Prevalence	0.3333	0.3333	0.3333
Detection Rate	0.3333	0.3333	0.2667
Detection Prevalence	0.3333	0.4000	0.2667
Balanced Accuracy	1.0000	0.9500	0.9000